Path concatenation: for every type A and every x, y, z there is a function from x = y and y = z to x = z carrying refl to refl

The Univalent Foundations Program

hott-book:basics.tex:local-snapshot-20261014; CC BY-SA 3.0 Unported

Editorial note. The counted statement is the author lemma labelled lem:concat (source lines 276-289); the proof span 293-324 is the author's first proof environment following it, retained line for line.

Editorial note. The author gives a second proof at source lines 326 onward; it is not counted and not retained on the card face.

Editorial note. The proof is self-contained: the auxiliary equation it cites is defined inside the retained proof itself.

Editorial scope. One original result of the HoTT book chapter cha:basics, counted once: transitivity of identity types (path concatenation) with its definitional conversion behaviour. Only the counted lemma and its first author proof are retained; the author's second proof and the surrounding commentary remain uncounted in the source.

Allowed standard prerequisites (editorial).

- The identity type of a type A and its reflexivity element refl x in x = x.
- The induction principle for identity types (path induction) with its conversion rule, as recapitulated in the book's own retained material.
- Dependent product types and type families: a family D over the paths of A assigning a type to each triple (x, y, p).

Lemma 0.0.1. *For every type A and every $x, y, z : A$ there is a function*

$$(x = y) \to (y = z) \to (x = z),$$

*written $p \mapsto q \mapsto p \cdot q$, such that $\mathsf{refl}_x \cdot \mathsf{refl}_x \equiv \mathsf{refl}_x$ for any $x : A$. We call $p \cdot q$ the **concatenation** or **composite** of p and q.*

First proof. The desired function has type $\prod_{(x,y,z:A)}(x = y) \to (y = z) \to (x = z)$. We will instead define a function with the equivalent type $\prod_{(x,y:A)}(x = y) \to \prod_{(z:A)}(y = z) \to (x = z)$, which allows us to apply path induction twice. Let $D : \prod_{(x,y:A)}(x = y) \to \mathcal{U}$ be the type family

$$D(x, y, p) :\equiv \prod_{(z:A)} \prod_{(q:y=z)} (x = z).$$

Note that $D(x, x, \mathsf{refl}_x) \equiv \prod_{(z:A)} \prod_{(q:x=z)} (x = z)$. Thus, in order to apply the induction principle for identity types to this D, we need a function of type

$$\prod_{x:A} D(x, x, \mathsf{refl}_x) \tag{0.0.2}$$

which is to say, of type

$$\prod_{(x,z:A)} \prod_{(q:x=z)} (x = z).$$

Now let $E : \prod_{(x,z:A)} \prod_{(q:x=z)} \mathcal{U}$ be the type family $E(x, z, q) :\equiv (x = z)$. Note that $E(x, x, \mathsf{refl}_x) \equiv (x = x)$. Thus, we have the function

$$e(x) :\equiv \mathsf{refl}_x : E(x, x, \mathsf{refl}_x).$$

By the induction principle for identity types applied to E, we obtain a function

$$d : \prod_{(x,z:A)} \prod_{(q:x=z)} E(x, z, q).$$

But $E(x, z, q) \equiv (x = z)$, so the type of d is (0.0.2). Thus, we can use this function d and apply the induction principle for identity types to D, to obtain our desired function of type

$$\prod_{x,y:A} (x = y) \to \prod_{z:A} (y = z) \to (x = z)$$

and hence $\prod_{(x,y,z:A)} (y = z) \to (x = y) \to (x = z)$. The conversion rules for the two induction principles give us $\mathsf{refl}_x \cdot \mathsf{refl}_x \equiv \mathsf{refl}_x$ for any $x : A$. □